

Un agente de código, sin nube

Las cuatro decisiones que lo hacen funcionar.

Costo por token: **cero**

Correctas: **3 / 3**

De costo por token a presupuesto

Sacar los modelos de pago de la ecuación no abarata el problema: lo **mueve de lugar**.

¿Cuánta VRAM hay?

¿Cuánto contexto entra?

¿Qué queda **para trabajar**?

Lo que manda es el contexto en t0

Los ejecutores maduros llegan al primer turno con su prompt de sistema, sus herramientas y los archivos del proyecto.

Antes de leer una línea del repo.

~20.500 tokens

de piso — y se reprocesan
en cada turno.

En la nube no se siente: sobran ventanas de 200K. En local es decisivo.

Elegido por descarte

Denso 7–14B — no convergen en tareas multi-paso.

Denso 27B en calidad usable — necesita 24 GB. No entra.

Denso 27B a 2 bits — entra, pero la cuantización se come el razonamiento de última milla.

MoE 35B con expertos en RAM — entra manteniendo 4 bits de calidad.

Para lo agéntico el MoE es el peor de los dos candidatos. Y el único que cabe.

llama.cpp con offload de expertos

Modelo en disco	20,8 GB
-----------------	---------

VRAM ocupada	11,9 / 16 GB
--------------	--------------

Generación	~53 tok/s
------------	-----------

Duplicar contexto a 64K	+300 MB
-------------------------	---------

Con la caché KV en 8 bits el contexto es barato. Racionarlo es un error caro: con 32K la corrida muere a mitad de tarea.

Que no la escriba el propio agente

Si los tests los escribe él, hay un modo de falla concreto:
**escribe su interpretación del enunciado en un test, pasa
su propio test, y queda verde estando mal.**

Calibrada contra un fallo conocido:
21 pasan, 2 fallan.

Y las 2 son la desviación real.

Tres corridas, tres correctas

Según especificación

3 / 3

Costo por token

~US\$1,07 → **cero**

Tiempo por corrida

~11 min → 7,5–16 min

El gate se verificó aparte en vez de crearle al agente: 45/45 tests, 0 lint, build limpio.

El orden de las decisiones importa

Primero el **arnés**, que define cuánto contexto queda. Después el **modelo** que entra en lo que sobra. Después el **serving**. Y la **aceptación** desde el principio.

Elegir el modelo primero es el error natural —
y el **más caro**.